

# Лекция 1

## Введение в экономику программной инженерии

Направление 09.03.04 «Программная инженерия»

Дисциплина: Основы экономики и управления  
программной инженерии

8 семестр, 2025/2026 учебный год

Преподаватель: Леонов Михаил Витальевич

# Терминология

- Программное обеспечение / программное средство — объект, состоящий из программ, процедур, правил и (если предусмотрено) сопутствующей документации и данных, относящихся к функционированию системы обработки информации (ГОСТ 28806-90).
- Программный продукт — программное средство, предназначенное для поставки, передачи, продажи пользователю (ГОСТ 28806-90).
- Информационная система ≈ «автоматизированная система» по ГОСТ 34, то есть система «люди + регламенты + данные + технические средства + ПО», обеспечивающая информационные процессы.
- Далее по курсу термин «программный продукт» используется как зонтичный термин для обозначения поставляемого и эксплуатируемого ИТ-решения
- Зачем? Единообразия изложения и потому, что экономические модели, рассматриваемые в курсе, оценивают затраты и риски именно на уровне «решения в эксплуатации», а не только на уровне исходного кода.
- Когда обсуждение требует строгого различия (например, юридические определения, договорные конструкции, лицензирование, требования к ИС как организационно-технической системе), будут использоваться конкретные термины.

# Структура лекции

1. Принятие экономических решений в программной инженерии
2. Особенности программного обеспечения как актива
3. Экономика жизненного цикла

# Принятие экономических решений

- Инженер создает программное обеспечение в условиях жестких ограничений: время, деньги, сотрудники
- SWEBOOK v4.0 (Гл. 12): «Экономика — это не бухгалтерия, это наука о принятии решений»
- Каждое решение — Python или Go, микросервисы или монолит — имеет свою цену и свои последствия
- Модель принятия решений:  
Потребности vs Предложения
- Потребности == Бизнес-цели (сократить время обработки заказа на 20%, масштабирование до 1 млн пользователей),
- Предложения == Техническая реализация (стек технологий, архитектурные паттерны, выбор алгоритмов)
- Если разрабатываемый программный продукт не решает потребности бизнеса или стоит дороже выгоды — это плохое инженерное решение.

# Критерии выбора

- Экономически эффективное инженерное решение — это то, которое отвечает специфическим целям заказчика в рамках выделенного бюджета
- Для коммерческих компаний: максимизация прибыли или стоимость бизнеса

По данным McKinsey, около 70% инициатив по цифровой трансформации в коммерческом секторе терпят неудачу в достижении своих целей. Причина часто в том, что технические решения отрываются от бизнес-целей максимизации прибыли.

- Для некоммерческих и госсектора: социальный эффект или минимизация затрат при заданном уровне качества и сервиса

Пример: сервис «Госуслуги». Его цель не «заработать на справках», а сократить бюджетные расходы на чиновников и повысить удобство граждан. Оцениваются не деньги, а KPI вроде «снижение времени ожидания в очереди» или «увеличение числа обработанных заявлений».

# Фундаментальная формула принятия решений

- Любое решение в программной инженерии сопровождается неопределенностью. Используем математическое ожидание!

$$EV = -Cost + \sum_{i=1}^n (P_i \times V_i)$$

$Cost$  — издержки,

$P_i$  — вероятность наступления сценария  $i$ ,  $i=1..n$ ,

$V_i$  — ценность (доход или убыток) в сценарии  $i$ ,  $i=1..n$ .

- Пример: у нас есть два варианта реализации защиты данных:
- Решение 1:** стоит 100 т.р., риск взлома 10%, ущерб при взломе 1 млн.р.  
 $EV = -100 + 0,1 * (-1\ 000) = -200$
- Решение 2:** стоит 150 т.р., риск взлома 1%, ущерб при взломе 1 млн.р.  
 $EV = -150 + 0,01 * (-1\ 000) = -160$
- SWEBOOK:** Альтернативные издержки - это выгода, которую вы упустили, выбрав данный путь. Когда вы выбираете одну технологию, вы автоматически отказываетесь от другой.

Пример: Если ваша команда полгода переписывала легаси-код вместо выпуска новой фичи, которую ждал рынок, ваши альтернативные издержки — это та прибыль, которую могла бы принести фича.

# Особенности ПО как актива (1)

- SWEBOOK: Программное обеспечение обладает свойством нематериальности

Свойства:

1. Отсутствие физического износа: код не ржавеет и не ломается от трения. Он может только «стареть» морально относительно меняющегося окружения (новые ОС, процессоры или требования рынка)
2. Нулевые предельные издержки: разрыв между капитальными затратами на разработку эталонного экземпляра и операционными расходами на его воспроизводство. Минимальные затраты на дистрибуцию последующих копий обуславливают значительный положительный эффект масштаба.
3. Высокая интеллектуалоемкость: структура себестоимости характеризуется отсутствием материальных затрат на сырье и ресурсы. Согласно статистике, ключевым драйвером стоимости выступает фонд оплаты труда с сопутствующими начислениями, доля которого составляет 70–85%.

# Особенности ПО как актива (2)

- **Гражданский кодекс РФ (Часть IV)**
- **Авторское право (ст. 1261):** авторские права на программы для ЭВМ (включая исходный текст и объектный код) охраняются так же, как авторские права на произведения литературы.
- **Исключительное право (ст. 1229):** правообладатель может по своему усмотрению разрешать или запрещать другим лицам использование результата интеллектуальной деятельности, а отсутствие запрета не считается разрешением.
- **Служебное произведение (ст. 1295):** если код создан в пределах трудовых обязанностей (служебное произведение), то по общему правилу исключительное право на него принадлежит работодателю, если договором не предусмотрено иное.
- **Срок охраны авторских прав:** исключительное право действует всю жизнь автора и 70 лет после его смерти (считая с 1 января года, следующего за годом смерти).



# Особенности ПО как актива (3)

- Экономика сетевого эффекта — это ситуация, когда полезность (и готовность платить) за программный продукт растёт по мере роста числа других пользователей, потому что продукт «работает через связи» между участниками сети.
- Эвристика Меткалфа:  $V \sim n^2$
- Прямой сетевой эффект: чем больше людей в одном и том же сервисе, тем ценнее он для каждого (мессенджеры).
- Перекрёстный (двусторонний) эффект характерен для платформ: больше пользователей привлекают больше поставщиков/разработчиков, что повышает ценность платформы и снова привлекает пользователей (маркетплейсы и экосистемы приложений).
- Почему «годами в убыток» — рационально?
  - В платформенной экономике рост сети запускает самоподдерживающийся цикл: больше участников → больше взаимодействий/выгоды → легче привлекать новых участников. Из-за этого компании могут сознательно инвестировать в рост базы (субсидировать одну сторону рынка, давать бесплатный доступ, снижать цены), потому что они наращивают долгосрочную ценность и рыночную власть, возникающую из сетевого эффекта.

# Налоговый аспекты

- Аккредитованные ИТ-организации (статус Минцифры) - выполнение требований по структуре профильной выручки (70% ИТ-доходов).
- Реестр российского ПО – используется для применения освобождения от НДС при реализации прав на конкретные программы/БД.
- На период 2025–2030 гг. льготная ставка налога на прибыль для ИТ-компаний — 5% (стандартная ставка 25%).
- Страховые взносы - пониженный тариф у ИТ-компаний — 15% с выплат работнику до предельной базы и 7,6% с суммы сверх предельной базы (стандартная ставка 30%).
- Освобождается от НДС передача исключительных прав и предоставление прав использования на программы для ЭВМ и базы данных, которые включены в единый реестр российских программ (включая обновления и доп. функционал, в том числе при удалённом доступе через интернет).

# Экономика жизненного цикла

- SWEBOOK: экономика жизненного цикла изучает общую стоимость программного продукта на протяжении всего времени его существования.
- Стандарт ISO/IEC/IEEE 12207 делит процессы на основные (разработка, эксплуатация, сопровождение) и вспомогательные. С точки зрения экономики, эти процессы делятся на:
  - инвестиционную фазу: (анализ, проектирование, кодирование) — деньги только тратятся,
  - эксплуатационную фазу: (внедрение, поддержка) — продукт начинает приносить ценность или экономить средства.
- Выбор модели жизненного цикла — это финансовое решение, влияющее на денежный поток.
- Waterfall (Каскадная модель):
  - Недостаток: высокая точка невозврата. Инвестиции заморожены до самого конца жизненного цикла. Если в конце выяснится, что рынок изменился, потери составят 100% бюджета.
  - Преимущество: прогнозируемость бюджета на ранних этапах (фиксированная цена).
- Agile (Итеративная модель):
  - Преимущество: ранний возврат инвестиций. Выпуская MVP (через 2 месяца, а не через 2 года), компания начинает окупать разработку гораздо раньше.
  - Гибкость: Возможность прекратить проект в любой момент, сохранив работающую часть функционала, снижение невозвратных издержек.

# «Закон Боэма» и стоимость исправлений

- Зависимость стоимости исправления дефекта от этапа жизненного цикла экспоненциальна

| Этап обнаружения ошибки | Относительная стоимость исправления |
|-------------------------|-------------------------------------|
| Сбор требований         | 1х                                  |
| Проектирование          | 3х – 5х                             |
| Кодирование             | 10х                                 |
| Тестирование            | 15х – 40х                           |
| Эксплуатация            | 60х – 100х                          |

- Качество — это не «эстетика кода», а финансовая стратегия управления рисками и затратами.
  - Почему мы тратим время на требования, ревью, тесты и автоматизацию, вместо того чтобы «быстрее пилить фичи»?
- Принцип Shift Left: переносить проверку качества как можно раньше, ближе к моменту внесения изменений, чтобы обратная связь была быстрой и дешёвой. Мы уменьшаем среднее время жизни дефекта (время от появления до обнаружения) и тем самым не даём ему «обрасти» зависимостями.

# Особенности поздних стадий

- Программный продукт перестаёт приносить ценность, но продолжает потреблять деньги и создавать риски, пока корректно не выведен из эксплуатации.
- Миграция данных
  - инвентаризация данных, выбор источника истины, правила соответствия, очистка, устранение дублей, контроль качества, тестовые прогоны, откат. Стоимость миграции растёт с объёмом данных, количеством интеграций и уровнем требований к непрерывности бизнеса.
- Архивирование и комплаенс
  - «Архив» как управляемый контур (политики хранения, доступ, журналирование, шифрование, регламенты уничтожения).
  - юридические обязанности не исчезают: данные и документы нужно хранить, уметь восстанавливать по запросу, обеспечивать разграничение доступа и аудит действий.
  - Отдельный риск — «забытые копии»: бэкапы, выгрузки, тестовые базы, выгрузки для подрядчиков; именно они чаще всего становятся источником утечек.
- Риски безопасности legacy
  - Legacy становится дорогим не потому что «старый код плохой», а потому что растут транзакционные издержки: дефицит компетенций, несовместимость с современными платформами, отсутствие обновлений, уязвимости.
  - Чем дольше живёт неподдерживаемое ПО, тем выше вероятность инцидента (утечка, остановка сервиса), а стоимость инцидента обычно нелинейна из-за каскада последствий (SLA, репутация, простои).

# Общая формула жизненных затрат

$LCC$

$$= C_{acquisition} + C_{operation} + C_{maintenance} + C_{disposal}$$

$C_{acq}$  — затраты на разработку или покупку;

$C_{op}$  — затраты на облака, электроэнергию, администрирование;

$C_{maint}$  — затраты на адаптацию к новым требованиям;

$C_{disp}$  — затраты на вывод из эксплуатации.

- LCC нужен, чтобы сравнивать альтернативы (купить/разработать; on-prem/cloud; монолит/микросервисы; импортное/отечественное) по полной стоимости владения, а не по цене разработки.
  - Решение — это не «утверждение сметы на разработку», а «утверждение будущего денежного потока на 3–7 лет».
- В современных корпоративных системах затраты на разработку ( $C_{acq}$ ) составляют лишь 20–30% от LCC, в то время как остальные 70–80% «съедает» многолетняя эксплуатация и поддержка.

# Полная стоимость владения

- управленческая модель, которой пользуются для сравнения вариантов и принятия решений
- Три ключевых блока:
  1. Прямые затраты на создание и приобретение  
ФОТ разработчиков, налоги, оплата сторонних сервисов и библиотек.
  2. Затраты на внедрение и инфраструктуру:
    - Внедрение — это момент, где ИТ становится операционным бизнесом: появляются счета за инфраструктуру, поддержка, обучение и риски простоя
  3. Затраты на сопровождение:
    - Исправительное (Corrective): багфиксы, инциденты, регрессии, поддержку L2/L3, аварийные релизы.
    - Адаптивное (Adaptive): изменение окружения — новые версии ОС/СУБД, требования регуляторов, изменения интеграций, импортозамещение, новые каналы.
    - Совершенствующее (Perfective): развитие, улучшение UX, ускорение процессов, новые фичи.
    - Профилактическое (Preventive): уменьшение будущих рисков — рефакторинг, техдолг, улучшение тестируемости, наблюдаемость, обновления зависимостей.

# Технический долг

- осознанное решение выбрать быстрое техническое решение сейчас, что влечет за собой дополнительные расходы в будущем.
  - «Тело долга»: время и деньги, необходимые для того, чтобы переписать код правильно (рефакторинг),
  - «Проценты по долгу»: дополнительное время, которое тратят разработчики на реализацию каждой новой функции из-за запутанности и плохого качества старого кода.
- Технический долг — это не всегда плохо.
  - если вам нужно первыми выйти на рынок, вы осознанно «берете кредит», чтобы захватить долю рынка. Прибыль от раннего релиза должна превышать будущие «проценты».

$$TD = \sum(Defects \times Repair\_Cost) + \sum(Debt\_Interest \times Time)$$

- Defects — количество (или оценённый объём) идентифицированных элементов технического долга: дефектов, архитектурных/кодовых несоответствий и иных технических несогласованностей, требующих устранения для достижения целевого уровня качества.
- Repair\_Cost — удельная стоимость устранения одного элемента долга, выраженная в трудозатратах или денежных расходах, включая работы разработки, тестирования и сопутствующие издержки внедрения.
- Debt\_Interest — «процентная ставка» технического долга: величина дополнительной стоимости (потерь производительности/дополнительных трудозатрат), возникающая при выполнении работ по развитию и сопровождению системы из-за наличия долга (например, рост трудоёмкости типовой задачи на X%).
- Time — временной горизонт, в течение которого система эксплуатируется при сохранении данного долга; при фиксированной ставке Debt\_Interest суммарная «переплата» по долгу возрастает пропорционально длительности этого периода.



# Дополнительное чтение

1. Подходы к расчету совокупной стоимости владения и эксплуатации комплексных систем безопасности —  
<https://habr.com/ru/companies/lanit/articles/566598/>
2. Сравниваем ТСО покупки «железа» и аренды облака —  
<https://habr.com/ru/companies/cloud4y/articles/425003/>
3. Своё железо или облако: считаем ТСО —  
<https://habr.com/ru/companies/cloud4y/articles/516456/>
4. Способы снижения совокупной стоимости владения гибридными и мультиоблачными средами —  
<https://habr.com/ru/companies/fgts/articles/590419/>
5. Как доказать что угодно при помощи ТСО —  
<https://habr.com/ru/articles/823344/>
6. Технический долг — тихий убийца <https://habr.com/ru/articles/797173/>
7. Как найти управу на технический долг—  
<https://habr.com/ru/companies/T1Holding/articles/888772/>
8. Техдолг. Большое руководство —  
<https://habr.com/ru/companies/banki/articles/903498/>
9. Честный разговор о техническом долге (в цифрах) —  
<https://habr.com/ru/companies/gazprombank/articles/985476/>
10. Тестирование влево, тестирование вправо: как не дать багам шанса —  
<https://software-testing.ru/library/testing/test-analysis/4363-shift-left-testing>